STREAM ALL IN ONE :

31 August 2025 21:24

```java
import java.util.*;
import java.util.stream.*;

public class StreamPractice {
    public static void main(String[] args) {

        // ----------------------------------------------------
        // Ways to Sort in Java (with Stream and Comparator)
        // ----------------------------------------------------

        List<String> list = new ArrayList<>();
        list.add("jatins");
        list.add("Kamniss");
        list.add("jotisss");
        list.add("munni");
        list.add("junni");
        list.add("kritika");
        list.add("bobyyyy");

        // 1. Natural Order (Ascending)
        List<String> sorted = list.stream()
                .sorted()   // natural order (A → Z)
                .collect(Collectors.toList());
        System.out.println("Natural order: " + sorted);

        // 2. Reverse Order
        List<String> sortedDesc = list.stream()
                .sorted(Comparator.reverseOrder()) // Z → A
                .collect(Collectors.toList());
        System.out.println("Reverse order: " + sortedDesc);

            // 4. Using Comparator.comparing()
        List<String> sortedByLength = list.stream()
                .sorted(Comparator.comparing(String::length))
                .collect(Collectors.toList());
        System.out.println("Sorted by length (comparing): " + sortedByLength);

        List<String> sortedByFirstChar = list.stream()
                .sorted(Comparator.comparing(s -> s.charAt(0)))
                .collect(Collectors.toList());
        System.out.println("Sorted by first char: " + sortedByFirstChar);

        // 5. Chained Sorting (ThenComparing)
        List<String> sortedByLengthThenName = list.stream()
                .sorted(Comparator.comparing(String::length)
                        .thenComparing(Comparator.naturalOrder()))
                .collect(Collectors.toList());
        System.out.println("Sorted by length then name: " + sortedByLengthThenName);

        // 6. Collections.sort() (Old Style)
```

```java
Collections.sort(list); // Ascending
System.out.println("Collections.sort ascending: " + list);
Collections.sort(list, Comparator.reverseOrder()); // Descending
System.out.println("Collections.sort descending: " + list);

// ----------------------------------------------------
// Difference between allMatch(), anyMatch(), and noneMatch()
// ----------------------------------------------------

List<Integer> nums = Arrays.asList(2, 4, 6, 8);
boolean allEven = nums.stream().allMatch(n -> n % 2 == 0);
System.out.println("allMatch all even? " + allEven);

List<Integer> nums2 = Arrays.asList(1, 3, 5, 8);
boolean anyEven = nums2.stream().anyMatch(n -> n % 2 == 0);
System.out.println("anyMatch any even? " + anyEven);

List<Integer> nums3 = Arrays.asList(1, 3, 5, 7);
boolean noneEven = nums3.stream().noneMatch(n -> n % 2 == 0);
System.out.println("noneMatch no even? " + noneEven);

// ----------------------------------------------------
// Java Stream filter() – Practical Examples
// ----------------------------------------------------

List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

// Filter even
List<Integer> evens = numbers.stream().filter(n -> n % 2 == 0).toList();
System.out.println("Evens: " + evens);

// Filter odd
List<Integer> odds = numbers.stream().filter(n -> n % 2 != 0).toList();
System.out.println("Odds: " + odds);

// Filter names examples
List<String> names = Arrays.asList("Jatin", "Kamni", "John", "Munni");
List<String> startsWithJ = names.stream().filter(n -> n.startsWith("J")).toList();
System.out.println("Starts with J: " + startsWithJ);

List<String> endsWithI = names.stream().filter(n -> n.endsWith("i")).toList();
System.out.println("Ends with i: " + endsWithI);

List<String> longNames = names.stream().filter(n -> n.length() > 4).toList();
System.out.println("Length > 4: " + longNames);

List<String> containsN = names.stream().filter(n -> n.contains("n")).toList();
System.out.println("Contains 'n': " + containsN);

// Filter null
List<String> withNull = Arrays.asList("apple", null, "banana", null, "cherry");
List<String> nonNull = withNull.stream().filter(Objects::nonNull).toList();
System.out.println("Non-null: " + nonNull);
```

```java
// Filter empty strings
List<String> empties = Arrays.asList("hi", "", "hello", "", "bye");
List<String> nonEmpty = empties.stream().filter(s -> !s.isEmpty()).toList();
System.out.println("Non-empty: " + nonEmpty);

// Distinct + filter
List<Integer> dupes = Arrays.asList(2, 4, 4, 6, 8, 8);
List<Integer> uniqueEven = dupes.stream().filter(n -> n % 2 == 0).distinct().toList();
System.out.println("Unique evens: " + uniqueEven);

// Filter > 5
List<Integer> greaterThan5 = numbers.stream().filter(n -> n > 5).toList();
System.out.println("Greater than 5: " + greaterThan5);

// Multiple conditions
List<String> result = names.stream()
        .filter(n -> n.length() > 4 && n.startsWith("J"))
        .toList();
System.out.println("Len>4 and startsWith J: " + result);

// Filter objects
class Student {
    String name;
    int marks;
    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}
List<Student> students = Arrays.asList(
        new Student("Jatin", 85),
        new Student("Kamni", 40),
        new Student("John", 92)
);
List<Student> passed = students.stream().filter(s -> s.marks >= 50).toList();
passed.forEach(s -> System.out.println("Passed: " + s.name));

// ---------------------------------------------------
// Map Operations
// ---------------------------------------------------

Map<Integer, String> map = new HashMap<>();
map.put(1, "Jatin");
map.put(2, "Kamni");
map.put(3, "Chaman");
map.put(4, "Ustad");
map.put(5, "Munni");

// 1. Iterate using stream
map.entrySet().stream().forEach(e -> System.out.println(e.getKey() + " => " + e.getValue()));

// 2. Filter by key
Map<Integer, String> keyGreaterThan2 = map.entrySet().stream()
        .filter(e -> e.getKey() > 2)
```

```java
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
System.out.println("Key > 2: " + keyGreaterThan2);

// 3. Filter by value
Map<Integer, String> nameWithN = map.entrySet().stream()
        .filter(e -> e.getValue().toLowerCase().contains("n"))
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
System.out.println("Names with n: " + nameWithN);

// 4. Convert values (to uppercase)
Map<Integer, String> upperCaseNames = map.entrySet().stream()
        .collect(Collectors.toMap(Map.Entry::getKey, e -> e.getValue().toUpperCase()));
System.out.println("Uppercase values: " + upperCaseNames);

// 5. Convert keys (*10)
Map<Integer, String> newKeys = map.entrySet().stream()
        .collect(Collectors.toMap(e -> e.getKey() * 10, Map.Entry::getValue));
System.out.println("Keys x10: " + newKeys);

// 6. Sort by key
Map<Integer, String> sortedByKey = map.entrySet().stream()
        .sorted(Map.Entry.comparingByKey())
        .collect(Collectors.toMap(
                Map.Entry::getKey, Map.Entry::getValue,
                (e1, e2) -> e1, LinkedHashMap::new));
System.out.println("Sorted by Key: " + sortedByKey);

// 7. Sort by value
Map<Integer, String> sortedByValue = map.entrySet().stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(
                Map.Entry::getKey, Map.Entry::getValue,
                (e1, e2) -> e1, LinkedHashMap::new));
System.out.println("Sorted by Value: " + sortedByValue);

// 8. Extract keys
List<Integer> keys = map.keySet().stream().toList();
System.out.println("Keys: " + keys);

// 9. Extract values
List<String> values = map.values().stream().toList();
System.out.println("Values: " + values);

// 10. Max value
Optional<Map.Entry<Integer, String>> maxValue = map.entrySet().stream()
        .max(Map.Entry.comparingByValue());
System.out.println("Max value: " + maxValue.get());

// 11. Count values > length 5
long count = map.entrySet().stream()
        .filter(e -> e.getValue().length() > 5)
        .count();
System.out.println("Count length>5: " + count);
```

```
        // 12. Group by length
        Map<Integer, List<String>> groupedByLength = map.values().stream()
                .collect(Collectors.groupingBy(String::length));
        System.out.println("Grouped by length: " + groupedByLength);
    }
}
```